



XPharmaCare

Motor de Retención y Rentabilidad

Metodología Completa del Sistema

Documentación Técnica y Operativa para Implementación

Autor: Braylin Alexander Jiménez Reynoso

Caso ficticio con datos simulados — Abril 2026
Pipeline: Python + R + Power BI (DAX)

© 2026 Braylin Alexander Jiménez Reynoso. Todos los derechos reservados.

1. Resumen Ejecutivo

El presente documento constituye la guía metodológica completa del Motor de Retención y Rentabilidad de XPharmaCare, un sistema de inteligencia artificial diseñado para el sector de retail farmacéutico de alto volumen. Su propósito es servir como referencia técnica y operativa para que cualquier profesional con conocimientos básicos de ciencia de datos pueda comprender, replicar e implementar cada componente del sistema.

El sistema no es un dashboard. Es un **pipeline de 9 componentes de Machine Learning conectados end-to-end** que procesa desde audio crudo del call center hasta alertas ejecutivas priorizadas por impacto financiero, sin intervención humana en la cadena de ejecución.

El diferenciador central del proyecto es el Modelado de Uplift con inferencia causal. Mientras que cualquier modelo de churn predice quién se va, el modelo de Uplift identifica a quién vale la pena retener, optimizando cada peso invertido en campañas de retención y evitando contactar a clientes que reaccionarían negativamente a la oferta.

1.1 Alcance del proyecto

El proyecto cubre el ciclo completo de inteligencia de clientes: desde la captura y procesamiento de datos no estructurados (audio) hasta la generación de acciones automáticas y su visualización en un dashboard de 5 páginas en Power BI. Los datos utilizados son simulados sobre un caso ficticio (XPharmaCare), pero la arquitectura, los modelos y la lógica de negocio son directamente transferibles a cualquier empresa de retail farmacéutico o de servicios con base de clientes recurrente.

1.2 Stack tecnológico

El sistema utiliza un stack polilingüe que aprovecha las fortalezas de cada lenguaje. Python se emplea para el procesamiento de datos, el feature engineering, los modelos supervisados y no supervisados, y la automatización del scoring diario. R se utiliza para la inferencia causal (Causal Forest del paquete grf) y el cálculo del índice de priorización, dado que su ecosistema estadístico es superior para este tipo de análisis. Power BI con DAX se emplea para la capa de visualización ejecutiva.

1.3 Cifras clave del caso

Métrica	Valor
Clientes en cartera	8,000
Transacciones procesadas	184,878
Ingreso bruto total	RD\$97.3M
SKUs activos	180 (82 crónicos)
Interacciones call center	5,864
Tasa de churn detectada	48.8%
Clientes persuadibles (Uplift)	2,822 (35.3%)
Sleeping Dogs identificados	2,150 (26.9%)
Costo de retención optimizado	RD\$86,525
Condiciones crónicas monitoreadas	6
Sucursales en la red	25 (17 con delivery)

2. Arquitectura del Sistema

El motor está diseñado como un pipeline secuencial donde la salida de cada componente alimenta al siguiente. Cada pieza es un notebook de Jupyter o un script de R independiente que puede ejecutarse de forma aislada para pruebas, o como parte del flujo completo en producción. La clave arquitectónica es que todos los componentes se conectan a través del campo `customer_id`, que actúa como llave universal en todo el sistema.

2.1 Flujo del pipeline

El flujo comienza con datos crudos (audio del call center, transacciones, datos clínicos) y termina en dos salidas: un CSV de alertas priorizadas y un dashboard de Power BI de 5 páginas. A continuación se describe cada componente en el orden en que se ejecuta:

#	Componente	Archivo	Lenguaje	Tipo
00	Speech-to-Text + Sentimiento	00_speech_sentiment_pipeline.ipynb	Python	IA / NLP
01	Modelo de Churn (entrenamiento)	01_modelo_churn_entrenamiento.ipynb	Python	Supervisado (GBM)
02	Scoring diario de Churn	02_modelo_churn_scoring_diario.ipynb	Python	Scoring automático
03	Clustering (entrenamiento)	03_segmentacion_clustering.ipynb	Python	No supervisado (K-Means)
04	Scoring de segmentos	04_segmentacion_scoring.ipynb	Python	Scoring automático
05a	Índice de Priorización	05_indice_priorizacion.R	R	Score ponderado + CLV
05b	Motor de Ofertas Automáticas	05_motor_ofertas_automaticas.ipynb	Python	Reglas de negocio
06a	Modelado de Uplift Causal	06_uplift_modelado_causal.R	R	Causal Forest (grf)
06b	Motor de Alertas	06_motor_alertas.ipynb	Python	Condicionales + CSV

Nota de implementación: Los componentes 01, 03, 05a y 06a se ejecutan una vez (entrenamiento). Los componentes 02, 04, 05b y 06b se ejecutan diariamente como jobs automatizados (cron). El componente 00 se ejecuta cada vez que se acumulan nuevos audios.

2.2 Modelo de datos (esquema estrella)

El sistema opera sobre un modelo estrella clásico de data warehouse. Las tablas de hechos registran eventos del negocio y las tablas de dimensiones contienen los atributos descriptivos de cada entidad. Todas se conectan a través de llaves foráneas.

Tablas de dimensiones

Tabla	Llave primaria	Campos principales
dim_clientes	customer_id	edad, sexo, provincia, zona, tipo_seguro, es_cronico, condicion_cronica, programa_lealtad, nivel_lealtad, canal_preferido, ingreso_estimado
dim_productos	product_id	nombre_producto, categoria, requiere_receta, precio_unitario, costo_unitario, margen_porcentaje, es_cronico
dim_sucursales	sucursal_id	nombre_sucursal, ciudad, provincia, tipo_sucursal, tiene_delivery, zona_cobertura
dim_fechas	fecha_id	fecha, anio, mes, trimestre, dia_semana, es_feriado

Tablas de hechos

Tabla	Volumen	Campos principales
fact_transacciones	184,878 registros	customer_id, product_id, sucursal_id, fecha_compra, cantidad, total_venta_dop, descuento_porcentaje, canal_compra, fue_receta_recurrente
fact_interacciones_callcenter	5,864 registros	customer_id, fecha_interaccion, motivo, sentimiento, puntaje_negativo, puntaje_positivo, estrellas, resuelto, tiempo_espera_seg, duracion_minutos
fact_adherencia_terapeutica	36,228 registros	customer_id, condicion_cronica, fecha_dispensacion, tasa_adherencia_periodo, dias_gap, ciclo_tratamiento
fact_campanas_retencion	8,000 registros	customer_id, grupo_experimento (Tratamiento/Control), tipo_oferta, hizo_churn_post_campana

La tabla `fact_campanas_retencion` es crítica para el modelo de Uplift. Contiene los resultados de un experimento A/B donde un grupo recibió ofertas de retención (Tratamiento) y otro no (Control). Sin esta tabla, el modelado causal no es posible.

3. Componente 00: Pipeline de Transcripción y Análisis de Sentimiento

Este componente es el punto de entrada del sistema. Su función es convertir datos no estructurados (audio del call center) en variables cuantitativas que alimentan directamente al modelo de churn. El proceso tiene dos etapas: transcripción de voz a texto (Speech-to-Text) y clasificación de sentimiento (Sentiment Analysis).

3.1 Transcripción con Whisper

Whisper es un modelo de reconocimiento de voz desarrollado por OpenAI, entrenado con 680,000 horas de audio en múltiples idiomas incluido el español. Se utiliza el modelo medium para equilibrar precisión y velocidad de procesamiento. El modelo recibe un archivo de audio (WAV, MP3, M4A, OGG, FLAC o WebM) y genera el texto transcrito.

Pseudocódigo del proceso de transcripción

```
CARGAR modelo_whisper = Whisper('medium')
PARA CADA archivo_audio EN carpeta_audios:
    texto = modelo_whisper.transcribir(archivo_audio, idioma='es')
    SI texto está vacío:
        marcar como ERROR y continuar
    GUARDAR (nombre_archivo, texto) en lista_resultados
```

3.2 Análisis de sentimiento con BERT

Una vez obtenido el texto, se clasifica el sentimiento usando un modelo BERT multilingüe preentrenado (nlptown/bert-base-multilingual-uncased-sentiment) disponible en HuggingFace. Este modelo fue entrenado con reseñas en múltiples idiomas y devuelve una clasificación de 1 a 5 estrellas, que el sistema agrupa en tres categorías:

Estrellas	Clasificación	Significado
1-2	NEGATIVO	Cliente frustrado, queja activa, problema sin resolver
3	NEUTRO	Consulta informativa, sin carga emocional clara
4-5	POSITIVO	Cliente satisfecho, agradecimiento, problema resuelto

Pseudocódigo de clasificación

```
CARGAR analizador = BERT_Sentiment('multilingual')
PARA CADA (nombre_archivo, texto) EN lista_resultados:
    resultado = analizador.clasificar(texto, max_longitud=512)
    estrellas = extraer_numero(resultado.etiqueta) // ej: '4 stars' → 4
    puntaje_positivo = resultado.confianza SI estrellas >= 4
    puntaje_negativo = resultado.confianza SI estrellas <= 2
    sentimiento = 'NEGATIVO' SI estrellas <= 2
    'NEUTRO' SI estrellas == 3
    'POSITIVO' SI estrellas >= 4
```

3.3 Variables generadas

El pipeline genera las siguientes variables a nivel de cada interacción, que luego se agregan a nivel de cliente para alimentar el modelo de churn:

Variable	Descripción
total_interacciones	Cantidad total de llamadas del cliente al call center
interacciones_negativas	Número de interacciones clasificadas como NEGATIVO
interacciones_positivas	Número de interacciones clasificadas como POSITIVO
puntaje_neg_promedio	Promedio de confianza en clasificación negativa (0 a 1)
puntaje_pos_promedio	Promedio de confianza en clasificación positiva (0 a 1)
estrellas_promedio	Promedio de estrellas (1-5) de todas las interacciones
ratio_negatividad	$\text{interacciones_negativas} / \text{total_interacciones}$
tasa_resolucion	Proporción de interacciones marcadas como resueltas
motivo_mas_frecuente	Categoría de queja más común del cliente

3.4 Integración con el sistema

El CSV generado se une con la tabla fact_interacciones_callcenter a través del nombre del archivo de audio (que en producción correspondería al ID de la interacción asignado por el sistema de telefonía). Esto permite que las columnas de sentimiento estén disponibles como features predictivas en el modelo de churn vinculadas al customer_id correspondiente.

Requisitos de infraestructura: Whisper requiere GPU para procesamiento eficiente (NVIDIA con CUDA). En CPU, el modelo medium procesa aproximadamente 1 audio por minuto. El modelo BERT es más ligero y puede correr en CPU sin problema.

4. Componentes 01-02: Modelo de Predicción de Churn

4.1 Objetivo del modelo

El modelo de churn calcula la probabilidad de que un cliente abandone sus compras recurrentes en los próximos 30 a 60 días. Esta probabilidad es el insumo principal del motor de ofertas y del sistema de alertas. No se trata de una clasificación binaria (se va / se queda), sino de una probabilidad continua entre 0 y 1 que permite graduar la intensidad de la intervención.

4.2 Feature Engineering

El proceso de construcción de features transforma los datos crudos de las tablas de hechos en un dataset analítico con una fila por cliente y múltiples columnas calculadas. Este dataset es el insumo tanto para el modelo de churn como para el clustering. Las features se agrupan en cuatro familias:

Familia 1: Comportamiento de compra

Estas variables capturan los patrones de consumo del cliente. Se calculan a partir de fact_transacciones, excluyendo los últimos 60 días (que se reservan para definir la variable objetivo).

Variable	Cálculo
total_transacciones	Conteo de transaction_id por cliente
total_gastado_dop	Suma de total_venta_dop por cliente
promedio_ticket_dop	Promedio de total_venta_dop por transacción
frecuencia_mensual	$\text{total_transacciones} / (\text{antigüedad_días} / 30)$
recencia_dias	Días desde la última compra hasta la fecha de corte
productos_distintos	Cantidad de product_id únicos comprados
canales_usados	Cantidad de canales de compra distintos
tendencia_gasto	$(\text{gasto_últimos_3m} - \text{gasto_3m_anteriores}) / \text{gasto_3m_anteriores}$
canal_dominante	Canal de compra más frecuente del cliente
descuento_promedio	Promedio de descuento_porcentaje aplicado
compras_receta_recorrente	Conteo de transacciones con receta recorrente

Familia 2: Sentimiento (del Componente 00)

Estas variables provienen del análisis de sentimiento del call center. Se agregan a nivel de cliente usando las interacciones previas a la fecha de corte. Las variables específicas se detallan en la sección 3.3 de este documento.

Familia 3: Adherencia terapéutica

Solo aplica a pacientes crónicos. Estas variables miden la disciplina del paciente en el cumplimiento de su tratamiento, lo cual es un predictor fuerte tanto de churn como de valor de vida.

Variable	Cálculo
tasa_adherencia_promedio	Promedio de tasa_adherencia_periodo de todos los ciclos
tasa_adherencia_min	Mínima tasa de adherencia registrada
dias_gap_promedio	Promedio de días entre recompras de medicamento
dias_gap_max	Máximo gap entre recompras
total_ciclos_tratamiento	Número total de ciclos de dispensación
ratio_retraso	Proporción de ciclos con gap > 7 días
tendencia_adherencia	Diferencia entre adherencia reciente y adherencia histórica

Familia 4: Perfil del cliente

Variables demográficas y de afiliación que provienen directamente de dim_clientes. Incluyen edad, sexo, zona geográfica, tipo de seguro, condición crónica, programa de lealtad, nivel de lealtad, canal preferido e ingreso estimado. Las variables categóricas se codifican con LabelEncoder para que el modelo pueda procesarlas.

4.3 Definición de la variable objetivo

Un cliente se define como churner si **no realizó ninguna compra en los últimos 60 días del dataset**. La fecha de corte se calcula como la fecha máxima del dataset menos 60 días. Todo el feature engineering se realiza con datos anteriores a esta fecha para evitar fuga de información (data leakage).

Importante: Los clientes sin historial de compras (recién registrados sin transacciones) se excluyen del entrenamiento para evitar falsos positivos.

4.4 Entrenamiento y selección del modelo

Se evalúan tres algoritmos de clasificación y se selecciona el de mejor rendimiento por AUC-ROC (Area Under the Receiver Operating Characteristic Curve):

Algoritmo	Hiperparámetros clave	Características
Logistic Regression	class_weight='balanced', max_iter=1000	Modelo base, interpretable, rápido
Random Forest	n_estimators=200, max_depth=10, class_weight='balanced'	Ensemble de árboles, robusto, poco sobreajuste
Gradient Boosting (GBM)	n_estimators=200, max_depth=5, learning_rate=0.1	Modelo ganador por rendimiento, captura no linealidades

Pseudocódigo del entrenamiento

```
DIVIDIR datos: 75% entrenamiento, 25% prueba (estratificado por churn)
ESCALAR features numéricas con StandardScaler
PARA CADA modelo EN [LogisticRegression, RandomForest, GradientBoosting]:
    entrenar(modelo, X_train, y_train)
    probabilidades = modelo.predict_proba(X_test)
    calcular AUC-ROC, precisión, recall, F1
SELECCIONAR modelo con mayor AUC-ROC
EXPORTAR modelo.pkl, scaler.pkl, encoders.pkl, feature_cols.pkl
```

4.5 Scoring diario (Componente 02)

El notebook 02 está diseñado para ejecutarse automáticamente cada día a las 6:00 AM mediante un cron job del sistema operativo. Su flujo es el siguiente:

```
CARGAR modelo_entrenado, scaler, encoders, feature_cols
CARGAR datos_frescos (dim_clientes, fact_transacciones, fact_callcenter, fact_adherencia)
RECONSTRUIR features con la misma lógica del entrenamiento
CODIFICAR categóricas con los mismos encoders (manejar valores nuevos)
GENERAR probabilidades = modelo.predict_proba(X)
CLASIFICAR riesgo: Muy Bajo (<20%), Bajo, Medio, Alto, Crítico (>80%)
EXPORTAR output_churn_scores.csv
REGISTRAR log de ejecución (fecha, clientes procesados, distribución)
```

El manejo de valores nuevos es crítico: si un cliente tiene un valor categórico que el modelo no vio durante el entrenamiento (por ejemplo, una nueva condición crónica), el sistema lo reemplaza por 'Desconocido' para evitar errores.

5. Componentes 03-04: Segmentación Conductual por Clustering

5.1 Filosofía del enfoque

La segmentación tradicional agrupa clientes por quiénes son (edad, género, ubicación). Este modelo los agrupa por **lo que hacen**. Se utiliza K-Means sobre 16 variables de comportamiento para descubrir patrones naturales en la cartera, sin imponer categorías predefinidas.

5.2 Variables de entrada

Se seleccionan exclusivamente variables de comportamiento (no demográficas) para que los segmentos reflejen patrones de acción:

- Compra: total_transacciones, total_gastado_dop, promedio_ticket_dop, frecuencia_mensual, recencia_dias, productos_distintos, canales_usados, descuento_promedio, tendencia_gasto, compras_receta_recurrente
- Sentimiento: total_interacciones, ratio_negatividad, estrellas_promedio
- Adherencia: tasa_adherencia_promedio, dias_gap_promedio, ratio_retraso

5.3 Proceso de selección de K

Se evalúan valores de K desde 2 hasta 10 usando dos métricas complementarias. El método del codo mide la inercia (suma de distancias al centroide) y busca el punto de inflexión donde añadir más clusters deja de reducir significativamente la inercia. El Silhouette Score mide qué tan bien separados están los clusters entre sí (valores cercanos a 1 indican buena separación). Se selecciona K=5 para obtener segmentos que sean accionables por el negocio.

Pseudocódigo

```
ESCALAR todas las features con StandardScaler
PARA K EN rango(2, 11):
    modelo = KMeans(n_clusters=K, n_init=10)
    etiquetas = modelo.fit_predict(X_escalado)
    calcular inercia y silhouette_score
SELECCIONAR K=5 (balance entre silhouette y accionabilidad)
ENTRENAR modelo_final = KMeans(K=5, n_init=20, max_iter=500)
EXPORTAR modelo_clustering.pkl, scaler_clustering.pkl
```

5.4 Segmentos resultantes y nombres de negocio

Cada cluster se perfila calculando los promedios de sus variables clave, y se le asigna un nombre de negocio basado en sus características dominantes:

Segmento	Clientes	Gasto prom.	Freq/mes	Crónicos	Adher.	Lealtad
Crónico Alto Valor	1,105	RD\$45.2K	2.68	100%	79%	67%
Crónico en Riesgo	1,430	RD\$18.7K	1.87	100%	77%	52%
Estándar Presencial	2,877	RD\$2.7K	0.52	2%	1%	31%
Esporádico / Inactivo	1,600	RD\$2.9K	0.50	4%	3%	33%
Frecuente Digital	988	RD\$63	0.06	21%	1%	37%

5.5 Scoring de nuevos clientes (Componente 04)

Cuando un cliente nuevo acumula 3 o más compras y tiene suficiente historial, el componente 04 lo clasifica automáticamente en uno de los 5 segmentos existentes. El proceso carga el modelo de clustering y el scaler guardados, calcula las features del cliente nuevo, las escala con el mismo scaler del entrenamiento, y asigna el cluster más cercano al centroide. El nombre del segmento se mapea desde un diccionario generado durante el entrenamiento.

6. Componente 05a: Índice de Priorización (Matriz de Valor)

Este componente, desarrollado en R, responde una pregunta crítica para el negocio: de todos los clientes en riesgo, ¿a quién salvamos primero? No todos los clientes generan el mismo valor. Un cliente que gasta RD\$500 al mes en pastillas para la presión no tiene el mismo impacto que uno que gasta RD\$50 en una crema. El índice de priorización cuantifica esa diferencia.

6.1 Componentes del índice

El índice se calcula como un promedio ponderado de cinco dimensiones, cada una normalizada en una escala de 0 a 100 usando min-max scaling:

Componente	Peso	Justificación
Tasa de adherencia terapéutica	30%	Métrica estrella: un crónico adherente tiene valor de vida garantizado por meses. Para no crónicos, se neutraliza en 50
Margen neto de compras	25%	Rentabilidad real, calculada cruzando transacciones con dim_productos (total_venta – costo_unitario × cantidad)
Ingreso bruto histórico	20%	Volumen total de negocio generado (total_gastado_dop)
Frecuencia de compra	15%	Reurrencia del comportamiento de compra (frecuencia_mensual)
Programa de lealtad	10%	Score escalonado: Platino=100, Oro=75, Plata=50, Bronce=25, Sin programa=0

Pseudocódigo del cálculo

```
CARGAR dataset_analitico, transacciones, productos
CALCULAR margen_netos = SUM(total_venta - costo_unitario * cantidad) POR cliente
NORMALIZAR cada componente de 0 a 100 con min-max scaling
indice = margen*0.25 + ingreso*0.20 + frecuencia*0.15 + lealtad*0.10 + adherencia*0.30
CATEGORIZAR:
  >= 75 → 'Platino (Top)'
  >= 55 → 'Alto Valor'
  >= 35 → 'Valor Medio'
  >= 15 → 'Valor Bajo'
  < 15 → 'En Riesgo de Pérdida'
```


6.2 CLV simplificado para crónicos

Para pacientes crónicos, se calcula un Customer Lifetime Value (CLV) simplificado con un horizonte de 24 meses. La fórmula utiliza el margen mensual promedio, la tasa de adherencia como proxy de retención, y una tasa de descuento del 1% mensual:

$$\text{margen_mensual} = \text{margen_neto_total} / \text{MAX}(\text{antiguedad_dias} / 30, 1)$$
$$\text{tasa_retencion} = \text{MIN}(\text{tasa_adherencia}, 0.98)$$
$$\text{CLV_24m} = \text{margen_mensual} \times (\text{tasa_ret} / (1 - \text{tasa_ret} + 0.01)) \times (1 - (\text{tasa_ret}/1.01)^{24})$$

El CLV promedio a 24 meses para crónicos resultó en RD\$990, con un Top 1% que supera los RD\$12,800. Este dato es crítico para calcular el impacto financiero de perder a cada cliente.

7. Componente 06a: Modelado de Uplift e Inferencia Causal

Este es el diferenciador central del proyecto. Es el componente que convierte un análisis de churn convencional en una herramienta de alto valor gerencial y es la razón por la que las empresas pagarían por este tipo de solución.

7.1 El problema que resuelve

La mayoría de los equipos de retención siguen un enfoque estándar: construyen un modelo de churn, identifican a los clientes de mayor riesgo, y les envían ofertas de retención. Este enfoque tiene un defecto fundamental: asume que todos los clientes en riesgo responden igual a la intervención. Eva Ascarza demostró en su paper «Retention Futility» (Journal of Marketing Research, 2018) que esto es falso. Existen cuatro tipos de clientes:

Tipo	% Cartera	Comportamiento	Acción correcta
Persuadible	35.3%	Se iba, pero la oferta lo retiene	INVERTIR — Son los únicos que vale la pena retener
Sure Thing	14.5%	Se queda con o sin oferta	NO GASTAR — Se quedan solos, el descuento es un regalo innecesario
Lost Cause	23.4%	Se va con o sin oferta	NO GASTAR — Ningún descuento los retiene
Sleeping Dog	26.9%	La oferta lo AHUYENTA	EVITAR CONTACTO — Contactarlos destruye valor

Sin modelado de Uplift, el 100% del presupuesto se gasta en todos los clientes en riesgo. Con Uplift, solo el 35.3% recibe inversión directa. El 26.9% de Sleeping Dogs no solo no debe recibir ofertas, sino que contactarlos los empuja a irse.

7.2 Base teórica: Causal Forest

Se utiliza el algoritmo Causal Forest del paquete grf (Generalized Random Forests) de Susan Athey e Ira Wager (Stanford), implementado en R. A diferencia de un Random Forest tradicional que predice resultados, el Causal Forest estima el efecto causal individual del tratamiento, conocido como CATE (Conditional Average Treatment Effect).

En términos simples: para cada cliente, el modelo responde la pregunta **«Si le doy la oferta, ¿cuánto cambia su probabilidad de quedarse versus si no se la doy?»**

Un CATE positivo significa que la oferta aumenta la retención (persuadible). Un CATE cercano a cero significa que la oferta no cambia nada (sure thing o lost cause, dependiendo de si el cliente churnea o no). Un CATE negativo significa que la oferta reduce la retención (sleeping dog).

7.3 Requisito de datos: el experimento A/B

El Causal Forest requiere datos de un experimento controlado (A/B test) para entrenar. La tabla `fact_campanas_retencion` contiene estos datos: un grupo de Tratamiento que recibió ofertas de retención y un grupo de Control que no recibió ninguna intervención. La asignación al grupo fue aleatoria.

Dato	Tratamiento	Control
Clientes	4,000 (50%)	4,000 (50%)
Tasa de churn	10.0%	18.9%
Reducción de churn	-47.1% vs control	Baseline

Si la empresa no tiene datos de un experimento A/B, debe ejecutar uno antes de implementar el modelado de Uplift. Sin grupo de control no hay causalidad, solo correlación.

7.4 Implementación paso a paso

Paso 1: Preparación de datos

```
W = 1 SI grupo_experimento == 'Tratamiento', 0 SI 'Control'  
Y = 1 SI NO hizo_churn_post_campana (retención), 0 SI churnea  
X = matriz de 17 features de comportamiento (mismas del churn)
```

Paso 2: Entrenamiento del Causal Forest

```
modelo_causal = causal_forest(  
    X = features,  
    Y = retencion (0/1),  
    W = tratamiento (0/1),  
    num.trees = 2000,  
    min.node.size = 15,  
    honesty = TRUE // Estimación honesta con muestra dividida  
)
```

El parámetro honesty=TRUE es crítico. Significa que el árbol usa una mitad de los datos para decidir dónde hacer los cortes (estructura) y la otra mitad para estimar los efectos (predicción). Esto evita el sobreajuste y produce estimaciones más confiables del efecto causal.

Paso 3: Estimación del CATE y clasificación

```
predicciones = modelo_causal.predict(X, estimate.variance=TRUE)
cate = predicciones.predictions // Efecto individual
cate_se = RAIZ(predicciones.variance) // Error estándar
```

```
CLASIFICAR:
SI cate > +0.05 → 'Persuadible'
SI cate < -0.03 → 'Sleeping Dog'
SI no churnea Y |cate| <= 0.05 → 'Sure Thing'
SI churnea Y |cate| <= 0.05 → 'Lost Cause'
```

7.5 Extensión a toda la cartera

Una vez entrenado con los datos del experimento, el modelo se aplica a la totalidad de los 8,000 clientes para estimar su CATE individual. Se agrega también un indicador de confianza basado en el error estándar: si el CATE es mayor que 2 veces su error estándar, la confianza es Alta; si es mayor que 1 vez, es Media; de lo contrario, es Baja.

7.6 Análisis de ahorro económico

Con un costo estimado de RD\$500 por oferta de retención, el análisis de ahorro compara tres escenarios de targeting para dimensionar el impacto financiero del Uplift frente al enfoque tradicional:

Escenario	Clientes contactados	Costo	Problema
Contactar a todos	8,000	RD\$4,000,000	Incluye 2,150 Sleeping Dogs que empeoran
Solo persuadibles (Uplift)	2,822	RD\$1,411,000	Ninguno — máxima eficiencia
Top riesgo (churn tradicional)	2,822	RD\$1,411,000	Mezcla lost causes y sure things

Ahorro neto por usar Uplift vs. contactar a todos: **RD\$2,589,000 DOP** (64.7% de reducción en costo de retención).

8. Componente 05b: Motor de Ofertas Automáticas

El motor de ofertas es el cerebro ejecutor del sistema. Toma las salidas de todos los componentes anteriores (churn, segmentación, priorización, uplift) y genera una acción específica para cada cliente sin intervención humana.

8.1 Inputs del motor

El motor cruza cuatro dimensiones por cada cliente, todas conectadas por customer_id:

- Probabilidad de churn (del componente 02): ¿Qué tan probable es que se vaya?
- Segmento conductual (del componente 04): ¿Qué tipo de cliente es?
- Índice de priorización y CLV (del componente 05a): ¿Cuánto vale?
- Tipo de respuesta Uplift (del componente 06a): ¿Responderá a la oferta?

8.2 Jerarquía de reglas de negocio

Las reglas siguen un orden jerárquico estricto. La primera condición que se cumple determina la acción, y las siguientes no se evalúan. La regla más importante es la de exclusión:

#	Condición	Acción	Descuento	Canal	Prioridad
0	Uplift = Sleeping Dog	No contactar	0%	Ninguno	Baja
1	Crónico AV + churn $\geq 70\%$	Descuento + Llamada VIP	15%	Llamada	Urgente
2	Crónico Riesgo + churn $\geq 50\%$	Recordatorio + Descuento receta	10%	WhatsApp	Alta
3	Frecuente + churn $\geq 60\%$	Puntos bonus + Oferta	8%	App Push	Alta
4	Buscador ofertas + churn $\geq 40\%$	Cupón exclusivo	12%	Email	Media
5	Digital + churn $\geq 40\%$	Envío gratis + Promo app	5%	App Push	Media
6	Crónico + churn $\geq 35\%$	Recordatorio receta + Desc.	7%	SMS	Media
7	Esporádico + churn $\geq 60\%$	Email reactivación	5%	Email	Baja
8	churn $< 20\%$	Sin acción (monitoreo)	0%	Ninguno	Monitoreo

Def	Cualquier otro caso	Comunicación estándar lealtad	3%	Email	Baja
-----	---------------------	-------------------------------	----	-------	------

8.3 Resultados del motor

Sobre los 8,000 clientes, el motor generó un costo total estimado de RD\$86,525, distribuyendo 2,571 comunicaciones estándar, 2,150 exclusiones de Sleeping Dogs, 1,836 clientes sin acción (bajo riesgo), 813 emails de reactivación, 298 recordatorios con descuento, 176 recordatorios de receta, 145 ofertas de puntos bonus, 10 llamadas VIP urgentes y 1 envío gratis con promo.

9. Componente 06b: Motor de Alertas Automáticas

El motor de alertas es la capa final del pipeline. Evalúa 14 condiciones de negocio contra umbrales configurables y genera un CSV con alertas priorizadas. Cada alerta contiene la fecha de generación, el tipo (churn, uplift, sentimiento, adherencia u operativo), la prioridad (Crítica, Alta, Media, Baja), un mensaje descriptivo, la métrica actual, el umbral que la disparó, la cantidad de clientes afectados, la acción recomendada y el impacto financiero estimado en DOP.

9.1 Umbrales configurables

Todos los umbrales se centralizan en un diccionario al inicio del notebook, lo que permite ajustarlos sin modificar la lógica del motor:

Parámetro	Valor	Significado
churn_critico_prob	0.80	Probabilidad mínima para riesgo crítico
churn_alto_prob	0.60	Probabilidad mínima para riesgo alto
clv_alto_valor_min	RD\$3,000	CLV mínimo para considerar alto valor
tasa_resolucion_meta	0.85	Meta del call center
ratio_negatividad_max	0.35	Máximo aceptable de interacciones negativas
adherencia_critica	0.60	Adherencia por debajo = intervención urgente
adherencia_baja	0.75	Adherencia por debajo = alerta media
dias_recencia_inactivo	180	Días sin compra = inactivo
costo_retencion_max	RD\$150,000	Presupuesto máximo mensual

10. Dashboard Power BI

La capa de visualización materializa toda la inteligencia del pipeline en 5 páginas diseñadas para dos perfiles de usuario: el gerente que necesita decisiones inmediatas (Centro de Mando, Clientes) y el estratega que planifica a mediano plazo (Productos, Motor de Retención, Pulso Operativo).

10.1 Página 1 — Centro de Mando

Vista ejecutiva con 6 KPIs principales (ingreso total, clientes activos, tasa de churn, CLV promedio, costo de retención, adherencia media), centro de alertas activas, gráfico de ingreso vs. tasa de churn (tendencia 11 meses), distribución de riesgo (donut de 5 niveles), clasificación Uplift y segmentación conductual.

10.2 Página 2 — Clientes

Tarjetas clickeables de los 5 segmentos con detalle interactivo de cada perfil, radar comparativo entre segmentos, matriz de valor por índice de priorización, y tabla de clientes de alto valor en riesgo crítico con la acción que el motor les asignó. Los filtros permiten cruzar por fecha, categoría de valor, riesgo de abandono, clasificación Uplift y segmento.

10.3 Página 3 — Productos

Ingreso por categoría (5 categorías: Medicamentos Crónicos 41.1%, Equipos Médicos 27.4%, Vitaminas 11.9%, OTC 10.5%, Cuidado Personal 9.1%), top 8 productos por ingreso, distribución por canal de venta (Delivery 31.4%, Sucursal 30.5%, App 21.4%, Web 16.7%), y tendencia de márgenes por categoría.

10.4 Página 4 — Motor de Retención

Matriz de Uplift visual con los 4 cuadrantes y su acción recomendada, tabla completa de las reglas del motor con costos y descuentos, y gráfico de efectividad de cada tipo de oferta comparando churn del grupo de tratamiento versus el grupo control (A/B test).

10.5 Página 5 — Pulso Operativo

Análisis de sentimiento del call center con distribución y tendencia temporal, top 6 motivos de fricción rankeados por severidad (delivery y agotamiento lideran con 23.6% de las quejas), y

adherencia terapéutica desglosada por las 6 condiciones crónicas monitoreadas (Hipertensión 81%, Diabetes T2 78%, Asma 76%, Dislipidemia 82%, Hipotiroidismo 84%, Artritis Reumatoide 73%).

10.6 Tema visual

El dashboard utiliza las fuentes Outfit (títulos, etiquetas y texto general) y JetBrains Mono (valores numéricos y KPIs) configuradas mediante un archivo JSON de tema personalizado para Power BI. La paleta de colores principal es verde (#00B48C) para métricas positivas, rojo (#EF4444) para alertas y riesgos, azul (#3B82F6) para datos informativos, y púrpura (#8B5CF6) para segmentos especiales.

11. Guía de Implementación

Esta sección describe los pasos necesarios para replicar el sistema completo desde cero en un entorno real.

11.1 Requisitos de software

Herramienta	Versión mínima	Uso
Python	3.9+	Pipeline principal (notebooks 00-06b)
R	4.2+	Inferencia causal y priorización (05a, 06a)
Jupyter Notebook	6.0+	Entorno de desarrollo y ejecución
Power BI Desktop	Última versión	Dashboard de visualización
NVIDIA GPU + CUDA	11.0+ (recomendado)	Aceleración de Whisper para audio

11.2 Librerías de Python

```
pip install pandas numpy scikit-learn matplotlib seaborn joblib
pip install openai-whisper transformers torch # Componente 00
```

11.3 Librerías de R

```
install.packages(c('grf', 'dplyr', 'readr', 'ggplot2', 'scales'))
```

11.4 Estructura de carpetas

```
proyecto/
├── audio/           # Archivos de audio del call center
├── data/            # CSVs de dimensiones y hechos
├── models/          # Modelos exportados (.pkl)
├── outputs/         # CSVs de salida
├── logs/            # Logs de ejecución
├── 00_speech_sentiment_pipeline.ipynb
├── 01_modelo_churn_entrenamiento.ipynb
├── 02_modelo_churn_scoring_diario.ipynb
├── 03_segmentacion_clustering_entrenamiento.ipynb
└── 04_segmentacion_clustering_scoring.ipynb
```

```
└─ 05_indice_priorizacion.R
└─ 05_motor_ofertas_automaticas.ipynb
└─ 06_uplift_modelado_causal.R
└─ 06_motor_alertas.ipynb
```

11.5 Orden de ejecución

La primera vez que se configura el sistema, los componentes deben ejecutarse en el siguiente orden estricto, ya que cada uno depende de las salidas del anterior:

1. Ejecutar 00_speech_sentiment_pipeline.ipynb (requiere audios en carpeta audio/)
2. Ejecutar 01_modelo_churn_entrenamiento.ipynb (genera modelo, scaler, dataset analítico)
3. Ejecutar 03_segmentacion_clustering_entrenamiento.ipynb (genera modelo de clustering)
4. Ejecutar 05_indice_priorizacion.R en RStudio o terminal (genera índice y CLV)
5. Ejecutar 06_uplift_modelado_causal.R (genera scores de Uplift)
6. Ejecutar 05_motor_ofertas_automaticas.ipynb (genera acciones)
7. Ejecutar 06_motor_alertas.ipynb (genera alertas)
8. Cargar CSVs de outputs/ en Power BI y conectar por customer_id

11.6 Automatización con cron

Una vez entrenados los modelos, los componentes de scoring se programan para ejecución diaria:

```
# Ejecutar a las 6:00 AM cada día
0 6 *** cd /ruta/proyecto && python 02_scoring.py >> logs/scoring.log 2>&1
5 6 *** cd /ruta/proyecto && python 04_cluster_scoring.py >> logs/cluster.log 2>&1
10 6 *** cd /ruta/proyecto && python 05_motor_ofertas.py >> logs/ofertas.log 2>&1
15 6 *** cd /ruta/proyecto && python 06_alertas.py >> logs/alertas.log 2>&1
```

12. Bibliografía

Ascarza, E. (2018). *Retention Futility: Targeting High-Risk Customers Might Be Ineffective*. *Journal of Marketing Research*, 55(1), 80–98.

Athey, S. y Wager, S. (2019). *Estimating Treatment Effects with Causal Forests*. *Journal of the American Statistical Association*, 114(523), 1–15.

Athey, S., Tibshirani, J. y Wager, S. (2019). *Generalized Random Forests*. *The Annals of Statistics*, 47(2), 1148–1178.

Radcliffe, N. y Surry, P. (2011). *Real-World Uplift Modelling with Significance-Based Uplift Trees*. *Portrait Technical Report TR-2011-1*.

Guelman, L., Guillén, M. y Pérez-Marín, A. (2015). *A Decision Support Framework for the Implementation of Uplift Modeling*. *European Journal of Operational Research*, 245(2), 502–510.

Friedman, J. H. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. *The Annals of Statistics*, 29(5), 1189–1232.

Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. y Sutskever, I. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. *OpenAI Technical Report*.

Devlin, J., Chang, M.W., Lee, K. y Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. *NAACL-HLT 2019*.

Organización Mundial de la Salud (2003). *Adherence to Long-Term Therapies: Evidence for Action*. Ginebra: OMS.

Fader, P. S. y Hardie, B. G. S. (2009). *Probability Models for Customer-Base Analysis*. *Journal of Interactive Marketing*, 23(1), 61–69.

Coussement, K. y De Bock, K. (2013). *Improving Customer Retention Management through Cost-Sensitive Learning*. *IESEG Working Paper Series 2013-MARK-01*.

13. Derechos de Autor

© 2026 Braylin Alexander Jiménez Reynoso. Todos los derechos reservados.

Este documento y su contenido, incluyendo la arquitectura del sistema, la metodología, los pseudocódigos, las reglas de negocio y el diseño del dashboard, son propiedad intelectual de su autor. Queda prohibida la reproducción total o parcial de este material sin la autorización expresa por escrito del autor.

El caso XPharmaCare es una empresa ficticia creada exclusivamente con fines demostrativos. Los datos utilizados son simulados y no representan información real de ninguna empresa existente. Cualquier similitud con empresas reales es coincidencia.

Las librerías y herramientas de código abierto referenciadas en este documento (scikit-learn, grf, Whisper, HuggingFace Transformers, entre otras) se distribuyen bajo sus respectivas licencias de código abierto y pertenecen a sus respectivos autores y organizaciones.

Para consultas sobre licenciamiento, colaboración o implementación comercial de esta solución, contactar directamente al autor.

Braylin Alexander Jiménez Reynoso
Economista de la Salud · Científico de Datos
Abril 2026